HACKATHON SUPPORTING DOCUMENTATION

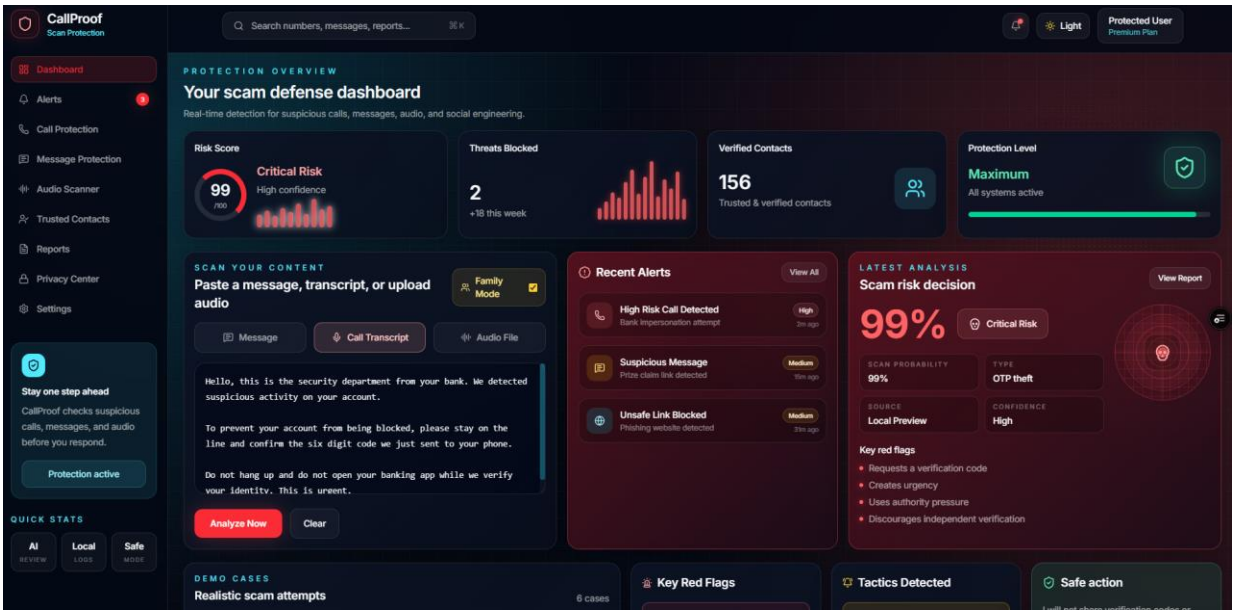
CallProof

A scam call and message detection dashboard that helps people pause, verify, and respond safely before a scam becomes a loss.

One sentence summary

CallProof analyzes suspicious messages, call transcripts, and uploaded audio to estimate scam risk, identify manipulation tactics, show the exact evidence behind the decision, and guide the user toward safer next steps.

Project type: Working hackathon MVP / web application
Current stack: Next.js, React, TypeScript, Tailwind CSS, OpenAI API, Vercel
Current inputs: Text message, call transcript, and audio upload with transcription
Primary value: Explainable scam detection with clear actions, not just a risk number



CallProof dashboard showing a critical-risk scam analysis state.

1. Executive summary

CallProof is built around one simple moment: the few seconds before a person clicks a link, shares a code, sends money, or trusts a caller who may not be who they claim to be.

Modern scams do not always look like obvious spam. Many now imitate banks, couriers, government offices, recruiters, family members, or investment platforms. They use urgency, fear, authority, secrecy, and emotional pressure to make people act before they can think clearly. That is the exact gap CallProof tries to fill. It does not ask the user to become a fraud expert. Instead, it gives them a practical safety layer that can read the message or call transcript, explain the risk, point to the suspicious evidence, and suggest a safe response.

The product is designed as a dashboard rather than a single warning popup because the user needs more than a binary answer. A suspicious call may contain multiple signals: a fake bank employee asking for a verification code, a delivery message containing a link and a payment request, or a family emergency text that pushes the user to transfer money quickly. CallProof turns those signals into a structured report: probability, risk level, attack type, red flags, manipulation tactics, evidence highlights, safe reply, and recommended next steps.

For the hackathon, the most important point is that this is not only a concept. The repository contains a functioning Next.js application with a rule-based detection engine, an optional AI semantic analysis layer, an audio transcription endpoint, demo scenarios, local scan history, report export, and a polished interface. The MVP is already strong enough to demonstrate the core user journey end to end.

Why this matters

Scams often succeed because the victim is rushed. CallProof creates a pause, gives the user a second opinion, and turns uncertainty into a safer decision.

2. Problem and user need

The core problem is not simply that scams exist. It is that scams are increasingly designed to exploit the exact moment when a person feels confused, afraid, rushed, or alone.

A typical user may receive a call from someone claiming to be from their bank, a message saying a package is blocked, or a text from a supposed family member using a new phone number. In each case, the attacker tries to push the user into immediate action. They may ask for a one-time code, request a small delivery fee, tell the user not to hang up, or create the feeling that legal or financial trouble is about to happen.

This is difficult for users because many warning signs are contextual. A single word like “urgent” may not prove anything, but “urgent” combined with a bank reference, a verification code request, and a warning not to call anyone is much more dangerous. People need a tool that can combine these signals, explain them, and give practical advice in plain language.

Main user pain points

Scams feel more realistic. Attackers copy the tone of real organizations and use familiar situations such as bank alerts, courier delays, tax notices, or family emergencies.

Victims are pushed to act quickly. Urgency and fear reduce the chance that a person will verify the situation through official channels.

Existing advice is too general. Users are often told “be careful online,” but in the moment they need specific guidance for the exact message or call they received.

Not every user is technical. Older adults, young users, and people under stress may need a simpler explanation and a safer suggested response.

Evidence matters. A risk score alone is not enough. Users need to see which phrases or patterns made the content suspicious.

Primary target users

User group	Need	How CallProof helps
Everyday mobile users	Check suspicious messages before replying	Paste the message and receive risk, evidence, and next steps.
Older or less technical users	Understand scams in plain language	Family Mode rewrites the result into simpler, calmer advice.
Families and caregivers	Help relatives verify suspicious calls	Exportable reports make it easier to share the analysis.
Small businesses	Reduce phishing and impersonation risk	Transcripts and messages can be reviewed before staff act.

3. Solution overview

CallProof works like a safety checkpoint for suspicious communication. It accepts the content, analyzes the risk, explains the evidence, and gives the user a safer next action.

The user can paste a suspicious text message, paste a call transcript, or upload an audio recording. Audio is transcribed first, then analyzed the same way as text. The analysis is designed to be explainable: it does not simply label something as “scam” or “not scam.” It tells the user what type of scam it appears to be, what tactics are present, and what specific pieces of the message raised concern.

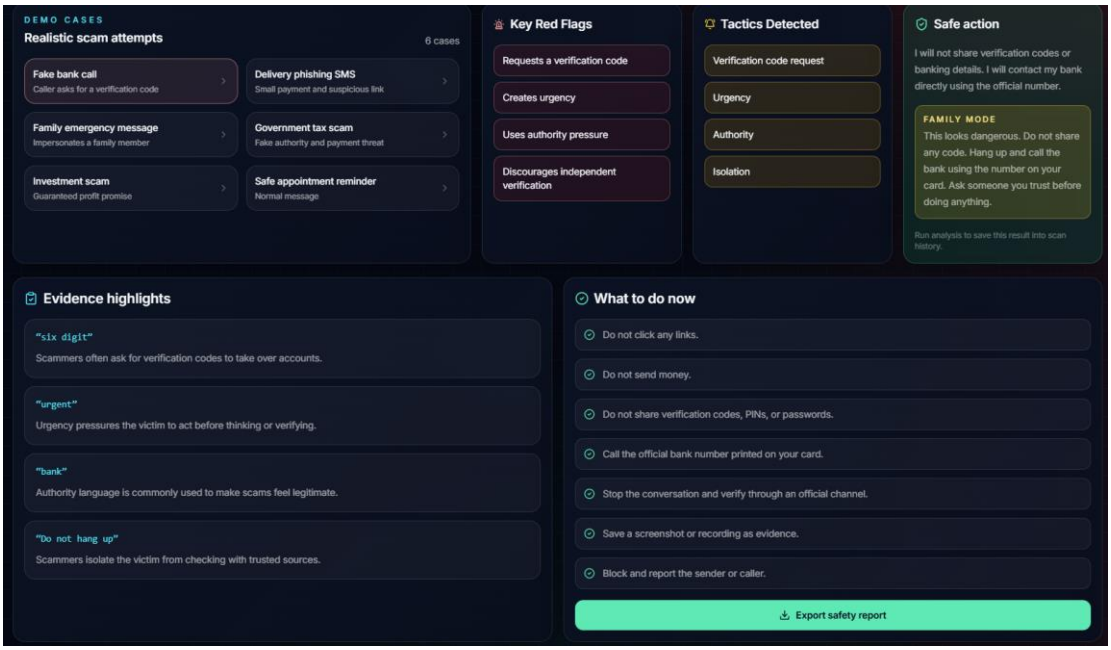
This makes the product useful even when the result is not perfectly certain. If a message is low risk, CallProof still reminds the user not to share passwords, PINs, card details, or verification codes. If the risk is medium, it recommends verification through official channels. If the risk is high or critical, it tells the user not to click, pay, share codes, or continue the conversation before checking independently.

Core product flow

Step	What happens	User value
1. Input	The user pastes a message, pastes a call transcript, or uploads audio.	The tool fits common real-life scam situations.
2. Transcription	Audio is converted into text through the transcription endpoint, with a fallback demo transcript if needed.	Calls can be checked without requiring manual typing.
3. Analysis	The system runs rule-based detection and, when configured, AI semantic review.	The result combines exact scam signals with contextual understanding.
4. Explanation	Risk score, attack type, tactics, red flags, and evidence are shown.	The user understands why the content is risky.
5. Action	The app generates next steps and a safe reply.	The user gets practical guidance, not just a warning.
6. Record	The result can be exported and recent scans are stored locally.	The user can review, share, or save evidence.

Design principle

The product should reduce panic, not increase it. Even when a scan is critical, the interface gives calm steps: stop, do not share information, verify through official channels, save evidence, and report or block the sender.



Evidence and next-step guidance are central to the product experience.

This evidence-first layout is important because trust is the hardest part of any safety product. A judge or user does not have to accept a black-box score; they can see the exact phrases and behaviors that caused the warning.

4. What makes CallProof different

The project is strongest when it is framed as an explainable decision assistant, not just a scam keyword checker.

Many simple scam checkers rely on obvious keywords or blacklisted links. That is useful, but it can miss the way social engineering actually works. CallProof is built around patterns: requests for verification codes, suspicious links, authority pressure, secrecy, payment requests, family emergency language, investment promises, and personal data requests. These are presented as red flags and tactics so the user can understand the attack rather than blindly trusting the score.

The optional AI layer adds semantic interpretation, while the deterministic engine gives the MVP reliability even without an API key. This is important for hackathon judging because the demo can still work in fallback mode. It also shows responsible product thinking: the AI improves the analysis, but the entire safety experience does not collapse if a third-party API call fails.

Feature highlights

Explainable risk scoring. The user sees probability, risk level, confidence, attack type, red flags, and manipulation tactics.

Evidence highlights. Suspicious phrases are extracted and connected to reasons, making the result easier to trust.

AI plus rule-based fallback. The app can use AI semantic review but still functions with deterministic scam detection.

Audio support. Users can upload recorded calls and analyze the resulting transcript.

Family Mode. The app turns technical risk into simpler advice for users who may need more support.

Exportable safety report. The result can be downloaded as a text report for evidence or later discussion.

Local scan history. Recent scans are saved in the browser without requiring a full user account system.

Demo-ready scenarios. The repository includes realistic cases such as fake bank calls, delivery phishing, family emergency scams, tax scams, investment scams, and a safe reminder.

Risk band	Interface meaning	Recommended behavior
0-29%	Low risk / green state	Proceed carefully, but never share sensitive codes or passwords.
30-79%	Suspicious / review state	Verify independently through official channels before responding.
80-100%	Critical / red state	Do not click, pay, share codes, or continue until verified.

5. Technical implementation

The MVP is implemented as a modern web application using Next.js, React, TypeScript, Tailwind CSS, Lucide icons, the OpenAI API, and Vercel deployment.

The application is organized around a clear separation between UI components, type definitions, demo cases, the deterministic scam engine, and API routes. The frontend handles the user flow: selecting input mode, loading demo cases, uploading audio, running analysis, showing results, exporting reports, changing theme, and maintaining local scan history. The backend API routes handle scam analysis and transcription when an OpenAI API key is configured.

Architecture summary

Layer	Implementation	Purpose
Frontend	app/page.tsx and reusable components	Controls the dashboard, input modes, results, history, report export, theme, and demo flow.
Rule engine	lib/scamEngine.ts	Detects patterns such as OTP requests, urgency, authority, links, payment pressure, and family emergency manipulation.
AI analysis	lib/aiAnalyzer.ts + app/api/analyze/route.ts	Uses structured AI output when configured and falls back to deterministic analysis when unavailable.
Transcription	app/api/transcribe/route.ts	Accepts audio files, validates size, sends audio for transcription, and returns transcript metadata.
Data model	lib/types.ts	Defines input modes, risk levels, attack types, tactics, evidence, and scan history structure.
Demo content	lib/demoCases.ts	Provides realistic examples for a quick judge-friendly demo.

Rule-based detection logic

The deterministic engine assigns scores to matched scam signals and then normalizes the result into a 0-99 scam probability. Signals include verification code requests, urgent language, authority pressure, isolation language, suspicious links, payment requests, family emergency pressure, unrealistic investment promises, and personal data requests. The engine also infers the most likely attack type, confidence, evidence, safe reply, next steps, and Family Mode advice.

AI analysis behavior

When an API key is present, the analysis route first creates a rule-based fallback result, then asks the AI layer to return a structured JSON analysis using the same schema. This approach keeps the output predictable and makes the AI layer safer to integrate because the application expects a controlled response shape. If the AI call fails, the fallback result is returned instead of breaking the product experience.

Audio handling

The transcription endpoint accepts an uploaded audio file, checks that a file exists, limits the file size to 25 MB, and sends the audio to the configured transcription model. If transcription fails during the frontend flow, the app uses a simulated transcript so the demo can still continue. For a hackathon setting, this is a practical choice because judges can see the intended audio flow even if API configuration or network conditions are imperfect.

MVP reliability choices

The project makes several practical MVP choices that are helpful in a time-limited hackathon. The analysis endpoint always creates a fallback result first, so the user receives a report even if the AI layer is unavailable. Recent scans are kept locally, which avoids building a full authentication and database layer too early. The demo cases are included directly in the app, which makes live presentation safer and repeatable.

These choices are not shortcuts around the core idea; they are the reason the product can be judged as a working prototype. The current version proves the safety workflow before adding heavier infrastructure such as accounts, persistent databases, organization dashboards, or production monitoring.

6. Demo narrative

Demo scenarios already included

Fake bank call: authority pressure and OTP theft risk.

Delivery phishing SMS: suspicious link and fee/payment pressure.

Family emergency message: emotional manipulation and urgent money request.

Government or tax scam: authority language and threat pressure.

Investment scam: unrealistic promises and “too good to be true” returns.

Safe appointment reminder: demonstrates that the product can produce a lower-risk result too.

History and education panels help the product feel like a safety dashboard, not just a one-time scanner.

The demo should close by returning to the human problem: the app is not trying to scare the user, but to give them enough context to stop and verify before acting.

7. Impact

Potential impact

If developed further, CallProof could become a lightweight safety layer for families, schools, community groups, small businesses, and individual mobile users. The most immediate impact is educational: after using the tool, a person can learn what urgency, isolation, payment pressure, suspicious links, and verification code theft look like in real messages. Over time, this can improve user judgment, not only block individual scams.

Responsible positioning

CallProof should be presented as a second-opinion safety tool. The safest message to users is: do not rush, do not share sensitive information, and verify through official channels.

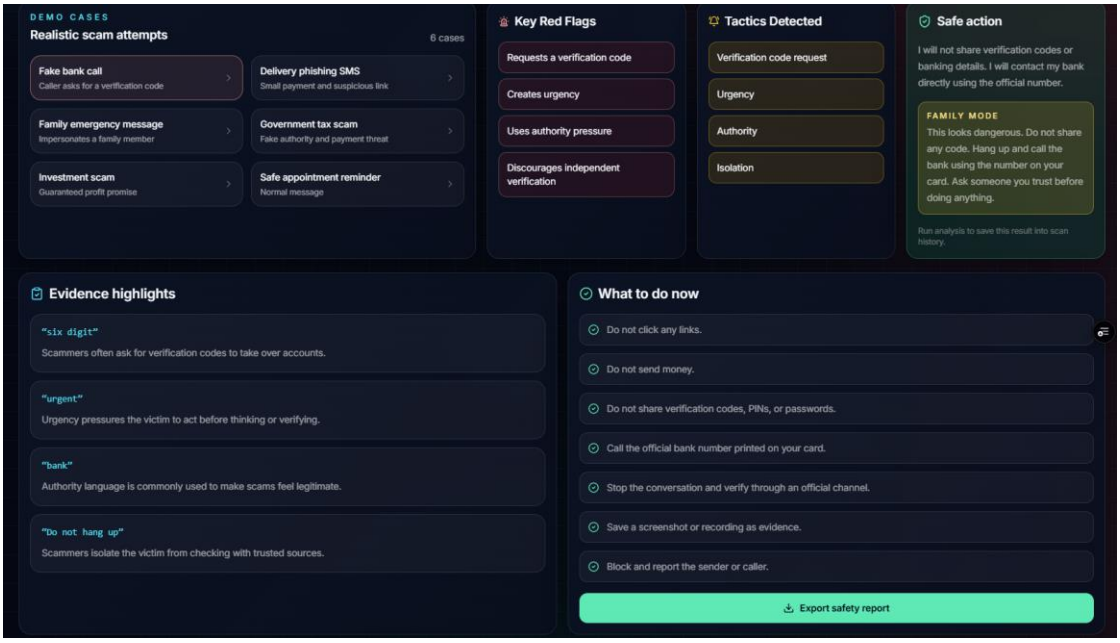
8. Roadmap and feasibility

The roadmap shows how it can become a more complete scam-prevention platform.

The current build proves the core loop: input, analysis, explanation, safe action, and report. The next step is to make the tool more integrated into real user behavior. Scams happen in phone calls, SMS threads, messaging apps, email inboxes, and web browsers, so future versions should reduce friction and meet the user where the risky interaction happens.

Longer-term product roadmap

Feature	Why it matters	Possible implementation
Live call monitoring	Scams often happen in real time before the user can paste a transcript.	Mobile app or call recording integration with real-time transcript alerts.
Phone number reputation checks	Caller identity is a major trust signal.	Integrate public reputation databases and user/community reporting.
Trusted contact verification	Family emergency scams rely on fake identity.	Allow users to verify emergencies through predefined trusted contacts.
Browser extension	Phishing links often arrive through messages and email.	Warn users before opening suspicious domains or payment pages.
Reporting integrations	Users need a simple way to report scams.	Export structured reports to banks, platforms, or cybercrime reporting channels.
Persistent accounts	Families and businesses may want shared history and alerts.	Optional user accounts with secure storage and permissions.



Risk breakdown view from the current MVP.